

COMPUTER VISION PROJECT

VISION CLI

README file

PROBLEM STATEMENT

- . Remote cloud servers and automated grading environments typically do not feature Graphical User Interface (GUI) desktop display managers.
- . Most computer vision applications are built upon fundamentally interactive window-rendering APIs such as OpenCV's `cv2.imshow()` function, which causes them to crash in a headless terminal environment.
- . Document processing utilities are not currently modular, and operate in isolation from one another, despite their complementary nature in performing both text extraction and spatial gesture key point processing.
- . A standalone non-GUI Python package is proposed for implementing OCR and hand gestures through command-line arguments.

SCOPE OF PROJECT

- . Input Array Processing: The validation and decoding of incoming static image files without raising unhandled exceptions.
- . Spatial Adaptive Binarization: The process of noise reduction using Gaussian filters and illumination compensation using adaptive Gaussian thresholding.
- . Optical Character Extraction: Parsing pre-processed image matrices into structured textual strings using PyTesseract. Hand Gesture Estimation: The processing of 21 spatial hand landmarks with MediaPipe to determine the count of raised fingers using joint coordinate vectors.

- . Modular Codebase: Separating the responsibility of the code into stand-alone packages for software testing and maintenance purposes.

TARGET AUDIENCE

- . Automated evaluation systems: software that does not require graphical user interface for the implementation of computer vision projects.
- . Data Processing Engineers: people who work in the terminal and execute batch scripts for transforming and extracting data from images.
- . Edge system developers: people who develop computer vision software for embedded Linux without a graphical user interface.

KEY FUNCTIONAL CAPABILITIES

- . Dynamic Task Selection: Run-time routing between text extraction, gesture analysis, or joint execution via --task CLI flags
- . Formatted Terminal Diagnostics: stdout logging of operational logs/output data - no display as pop-up windows
- . Automated Shape Validation: PyTest unit tests to ensure image array integrity before processing by the module.

SYSTEM OVERVIEW

- . The Automated Optical Document & Gesture-Based Smart Reader is an enterprise-level, command-line computer vision engine written in Python.
- . The system performs Optical Character Recognition and hand gesture landmark tracking on static image assets within headless terminal environments.
- . A Pipe-and-Filter design pattern is implemented to process arrays of input sequentially through independent functional modules.

CORE OPERATIONAL MODULES

- . main.py: Command-line interface parser and system entry point.
- . src/logger.py: Centralized logging engine providing execution diagnostics to stdout.
- . src/image_loader.py: Input validation layer enforcing file existence and matrix decoding checks.
- . src/preprocessor.py: Image binarization engine combining Gaussian blurring and adaptive thresholding.
- . src/ocr_engine.py: PyTesseract wrapper converting binary matrices into clean text strings.
- . src/gesture_detector.py: MediaPipe keypoint engine mapping 21 spatial hand joint coordinates.
- . tests/test_pipeline.py: PyTest verification suite testing matrix array shapes.

SYSTEM REQUIREMENTS AND DEPENDENCIES

- . Python Engine: Python version 3.9 or higher.
- . System Tesseract Dependency: Tesseract-OCR binary engine installed on the host system.
- . Python Packages: Listed in requirements.txt (opencv-python, mediapipe, pytesseract, numpy, pytest).

INSTALLATIONS AND SETUP INSTRUCTIONS

Step1 – Repository cloning

Step2 – Virtual environment setup

Step3 – Dependencies installations

EXECUTION AND SYNTAX ARGUMENTS

- . Full pipeline execution
- . Text extraction mode only
- . Gesture tracking mode only

UNIT TESTING PROCEDURE

- . Execute automated array validation test by running.

